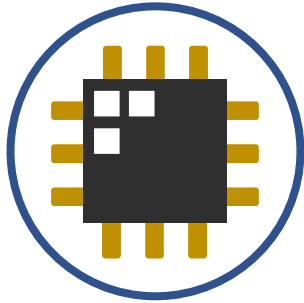




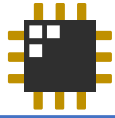
EEE 113:

Introduction to Electrical and Electronics Engineering Systems

Week 2 – Multiplexers and Decoders

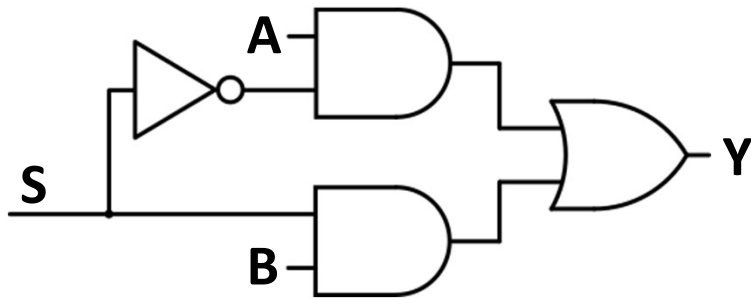
1

Today, we'll be diving into combinational logic blocks. Combinational logic blocks are fundamental components of digital systems. They are built using basic logic gates like AND, OR, and NOT gates, and they perform specific functions based on the combination of their inputs. For this discussion, we'll be focusing more on multiplexers and decoders.



Remember this?

- We can make combinations of logic gates to make more complex logic!

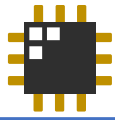


$$Y = S'A + SB$$

Inputs			Out
S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

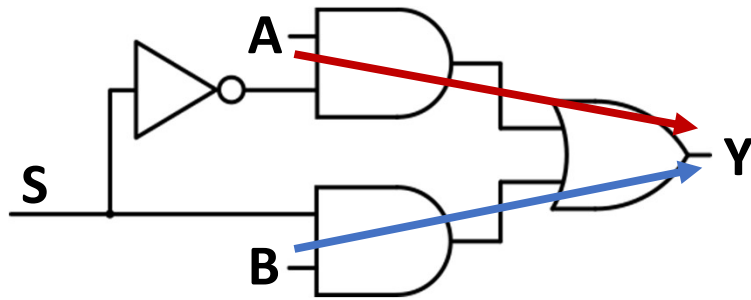
2

This was one of the complex logic that we analyzed last week. We started from the circuit and then derived the Boolean expression corresponding to that circuit. Then, we derived the truth table so better explain the functionality.



Multiplexers

- Multiplexer (mux) is a circuit that selects data from *one of many* (2^n) inputs and directs it to a *single* output using an n-bit selector



If $S = 0$, $Y = A$

If $S = 1$, $Y = B$

$$Y = S'A + SB$$

Inputs			Out
S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

3

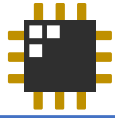
If you can remember how we explained the functionality of the circuit, we identified that the output basically just copies the value of A whenever S is 0. If S is 1 on the other hand, the output copies the value of B. You can observe this from the truth table on the right. Highlighted in red are the rows corresponding to $S = 0$, and highlighted in blue are the rows corresponding to $S = 1$. Taking a close look at the behavior of Y relative to the inputs, we see that Y just copies whatever A is in the red region, and Y copies whatever B is in the blue region. This reiterates the high-level behavior that we observed.

High-level description:

If $S = 0$, select A (Y copies the value of A)

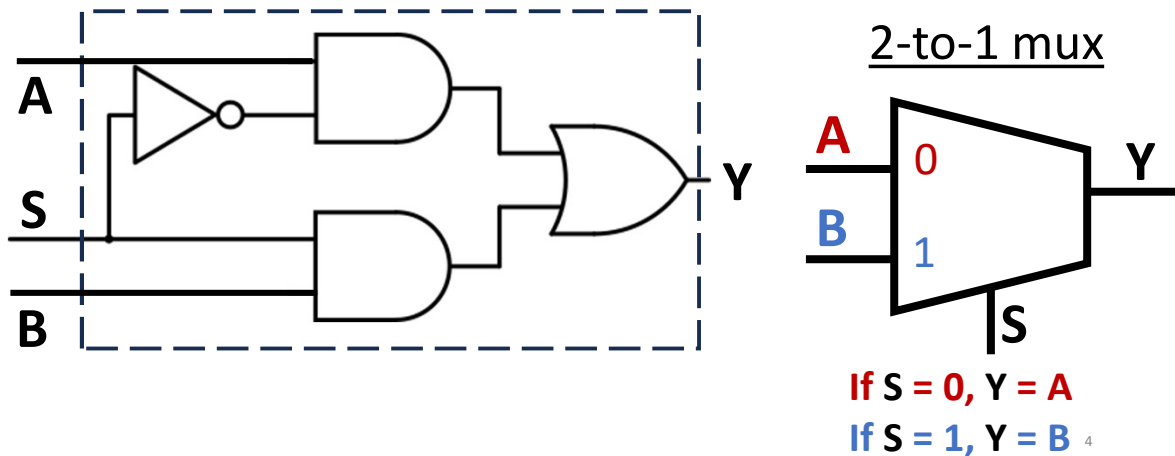
If $S = 1$, select B (Y copies the value of B)

This logic block is called a multiplexer, which selects from one of many (2^n) inputs and directs it to a single output using an n-bit selector.

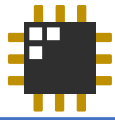


Multiplexers

- Multiplexer (mux) is a circuit that selects data from *one of many* (2^n) inputs and directs it to a *single* output using an n-bit selector



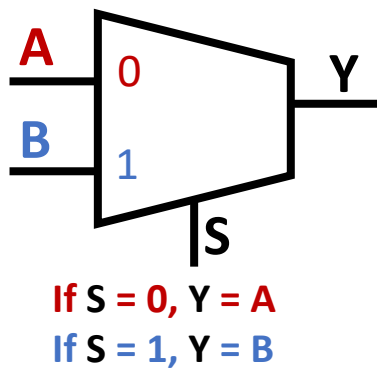
Now, since we have identified the unique functionality of this circuit, we can lump all of those logic gates together as a single box. When considering logic blocks, we tend to increase the level of abstraction and not show the component blocks anymore. At this point, given that we know what the functionality of the block is, we don't have to draw the corresponding logic gate equivalent circuit anymore. This simplifies things as we implement more complex logic.



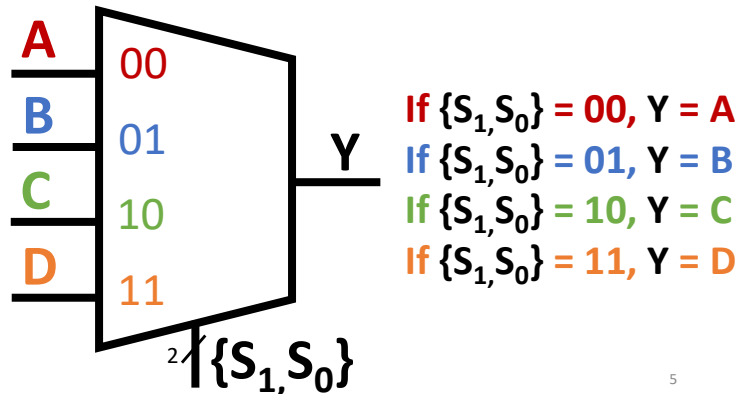
Multiplexers

- Multiplexer (mux) is a circuit that selects data from *one of many* (2^n) inputs and directs it to a *single* output using an n-bit selector

2-to-1 mux



4-to-1 mux



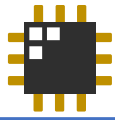
5

The functionality of a multiplexer is not limited to only two inputs. We can extend the concept by increasing the number of selector bits (the bottom lines) as shown in the diagram. Since we identified earlier that the S signal basically selects which of the two inputs will pass through the output, we can extend that concept by using two bits for the S signal, now S_1S_0 , which makes the multiplexer able to select between four different inputs, corresponding to the different S_1S_0 combinations. This now becomes a 4-to-1 multiplexer, as it selects 4 different inputs to map to a single output.

1-bit selector, 2^1 inputs

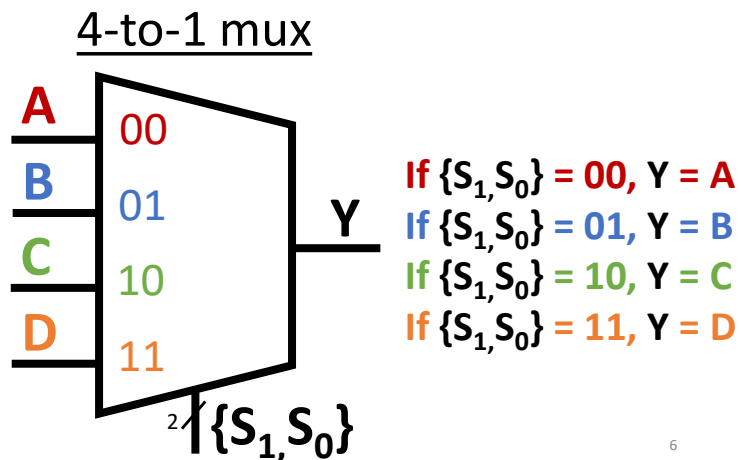
2-bit selector, 2^2 inputs

The inputs being selected corresponding to the different S_1S_0 combinations are shown with their color correspondence for easy identification for now.



Multiplexers

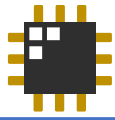
- Write the equivalent Boolean expression for Y.



So, just a practice. While I said earlier that once we move on to logic blocks we tend not to draw the logic gate equivalent circuit anymore. However, it is always nice to still know what is happening in the backend as we explore more complex blocks. So now, given the functionality of the 4-to-1 multiplexer, which is shown on the right, what is the equivalent Boolean expression for the output.

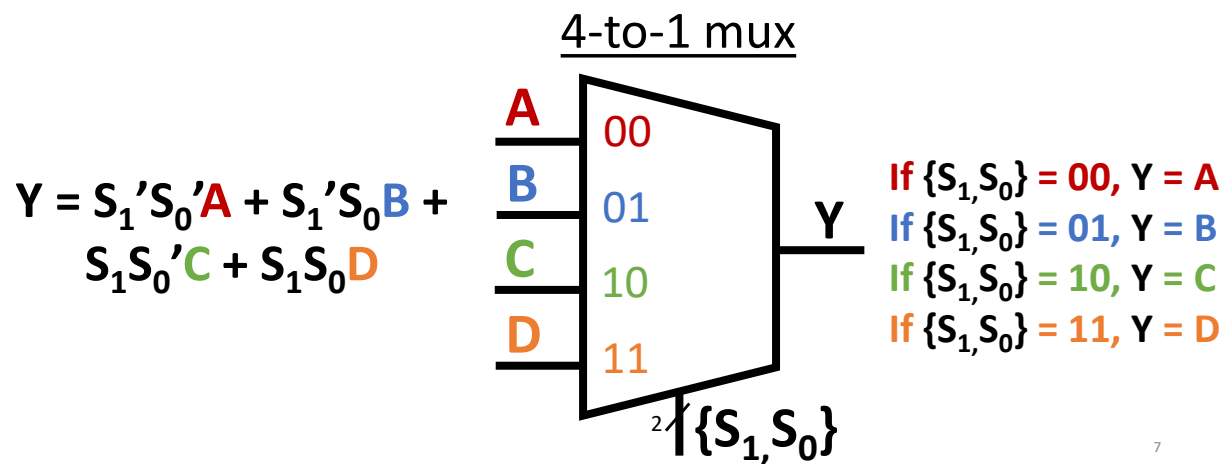
Note how different S_1S_0 combinations map one of the inputs to the output. For example, if S_1S_0 is 00, then Y just copies whatever A is. This implies the Boolean expression $S_1'S_0'A$. Do this for all other combinations of S_1S_0 and their corresponding inputs and we will end up with.

$$Y = S_1'S_0'A + S_1'S_0B + S_1S_0'C + S_1S_0D$$

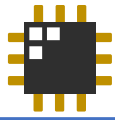


Multiplexers

- Write the equivalent Boolean expression for Y.

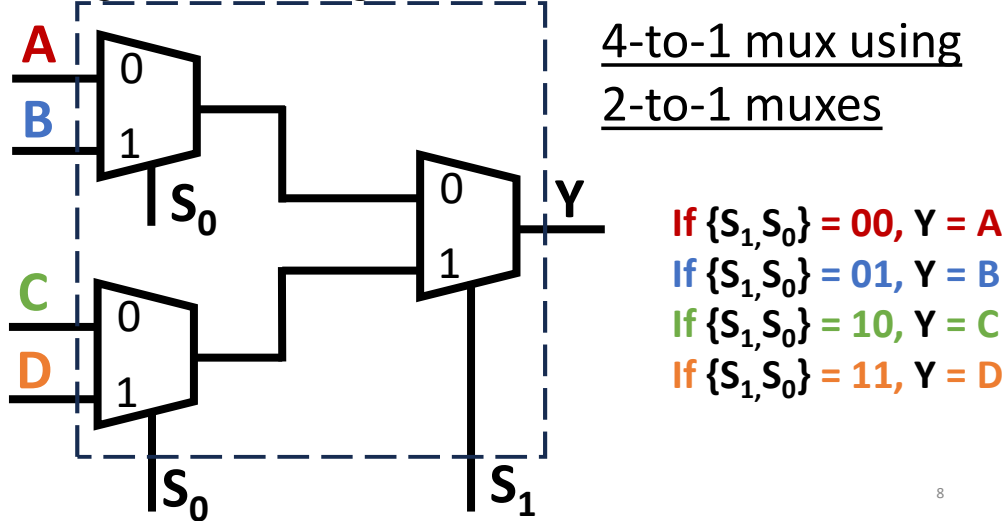


This is the Boolean expression for the output of a 4-to-1 multiplexer.



Combining multiplexers

- You can make larger muxes using smaller muxes!



The cool thing about multiplexers is that we can create larger multiplexers using smaller ones. In this diagram, we have a 4-to-1 multiplexer being implemented using 2-to-1 multiplexers.

We can verify the functionality by looking at how the subblocks behave given the inputs. For instance, if S_0 and S_1 are both 0, then the multiplexers on the right would be selecting the A input and the C input at the same time, while the multiplexer on the right would be selecting the one corresponding to the 0 input, which was A as we identified earlier. Thus, we can say that for $S_1S_0 = 00$, then Y just copies A, which is the correct behavior of the 4-to-1 multiplexer.

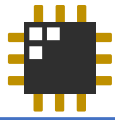
We can extend this description for all the other cases.

For $S_1S_0 = 01$: The multiplexers on the right would be selecting the B input and the D input at the same time, while the multiplexer on the left would be selecting the one corresponding to the 0 input, which was B as we identified earlier. Thus, we can say that for $S_1S_0 = 01$, then Y just copies B, which is the correct behavior of the 4-to-1 multiplexer.

For $S_1S_0 = 10$: The multiplexers on the right would be selecting the A input and the C input at the same time, while the multiplexer on the left would be selecting the one corresponding to the 1 input, which was C as we identified earlier. Thus, we can say that for $S_1S_0 = 10$, then Y just copies C, which is the

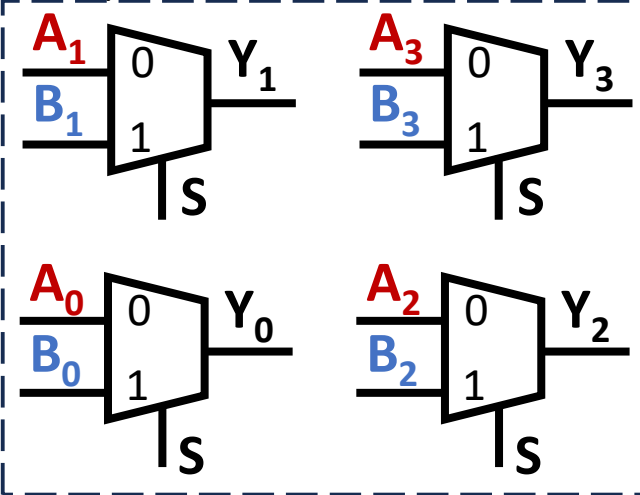
correct behavior of the 4-to-1 multiplexer.

For **S1S0 = 11**: The multiplexers on the right would be selecting the B input and the D input at the same time, while the multiplexer on the left would be selecting the one corresponding to the 1 input, which was D as we identified earlier. Thus, we can say that for S1S0 = 11, then Y just copies D, which is the correct behavior of the 4-to-1 multiplexer.



Combining multiplexers

- Multiplexers can be used in parallel to handle several bits at once!



4-bit 2-to-1 mux

If $S = 0$, $\{Y_3, Y_2, Y_1, Y_0\} = \{A_3, A_2, A_1, A_0\}$

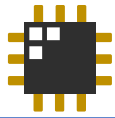
If $S = 1$, $\{Y_3, Y_2, Y_1, Y_0\} = \{B_3, B_2, B_1, B_0\}$

All muxes here *run* in parallel

9

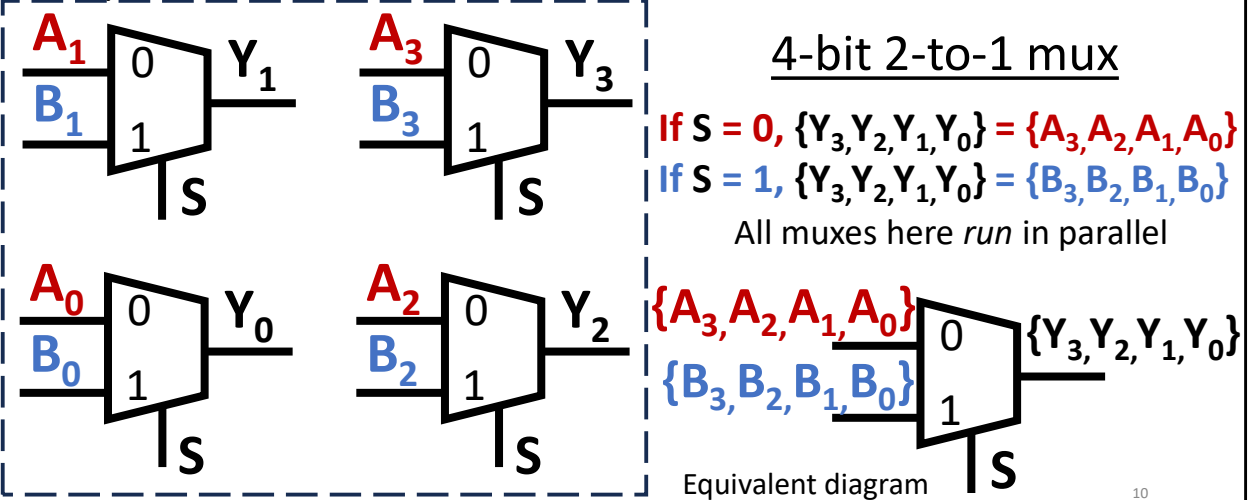
Multiplexers can also select several bits at once by having them in parallel. Note that the multiplexers we explored earlier only had 1-bit inputs for A,B (and C,D for 4-to-1 mux). Now, if we want to extend this, then we just create several instances of the same multiplexer and have each of them handle individual bits of our binary string.

For example, this diagram shows how a 1-bit 2-to-1 mux, that we already know, can be extended as a 4-bit 2-to-1 mux when there's four of them running in parallel, each multiplexer handling a separate bit. In this example, if S is 0, all the multiplexers will be selecting the inputs A_x to be passed to the output Y_x . Then, if S is 1, all the multiplexers will be selecting the inputs B_x to the output Y_x . This is how we implement a multi-bit multiplexer.

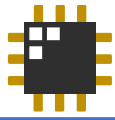


Combining multiplexers

- Multiplexers can be used in parallel to handle several bits at once!



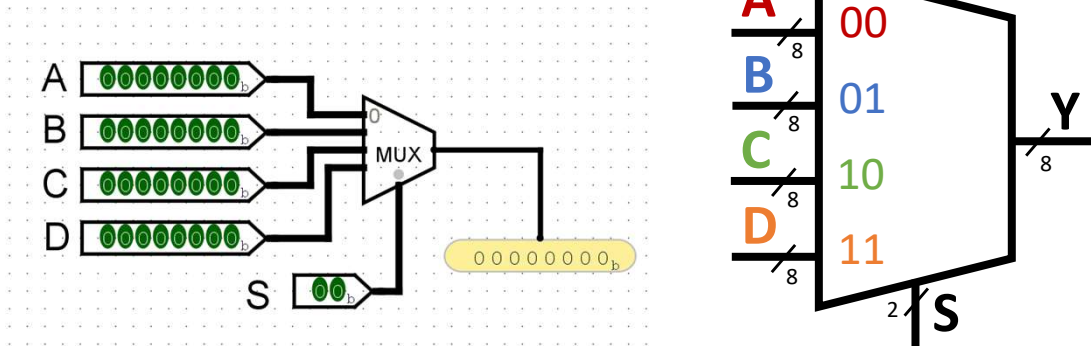
And here's the equivalent diagram for that one. We can lump all of those multiplexer subblocks and just draw a single multiplexer that indicates that it can handle several bits at the same time.



Multiplexers

- Multiplexers can be used in parallel to handle several bits at once!

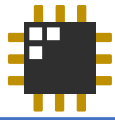
8-bit 4-to-1 mux



When considering multi-bit muxes, we sometimes don't put the number of bits to make the drawing simpler.

Here's a demonstration of the 8-bit 4-to-1 multiplexer. Note that for simplicity, we don't try to write all the bits anymore in the multiplexer diagram, since that would end up being too long, especially if you have a lot of bits to be considered in parallel. In this case, we assume that A, B, C, D, and Y are all 8-bits long, and therefore the multiplexer should be an 8-bit 4-to-1 multiplexer, without explicitly writing all the 8 bits. Moreover, since we can see that the multiplexer has 4 inputs, then we can say that S is 2-bits, without explicitly writing how many bits it is, since it has to select one of the inputs to be passed to the output.

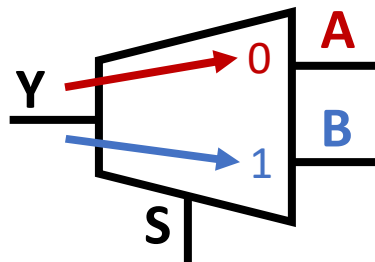
As you can see in the demo, different values of A, B, C, and D were selected depending on what the value of S is. The input corresponding to S=00 is still the input A (at the top) in this demo and the assignment just continues downward in increasing binary value. This is consistent with the multiplexer drawing on the right.



Demultiplexers

- We can technically do the reverse functionality!

Depending on the selector, direct Y to A or direct Y to B



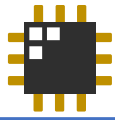
Inputs		Outputs	
S	Y	A	B
0	0	0	0
0	1	1	0
1	0	0	0
1	1	0	1

Note: Y is the input here (S is still an input), A and B are outputs

12

We can technically reverse the functionality of a multiplexer and have a block that does that. In this diagram, Y is now being passed to either A or B, depending on the selector S. Note that in this case, Y is now the input of the logic block while A and B are the outputs. S remains an input of the block as it has to select the mapping that needs to be performed. If S is 0, whatever the value of Y is will be passed to the output A. If S is 1, then it is mapped to the output B instead.

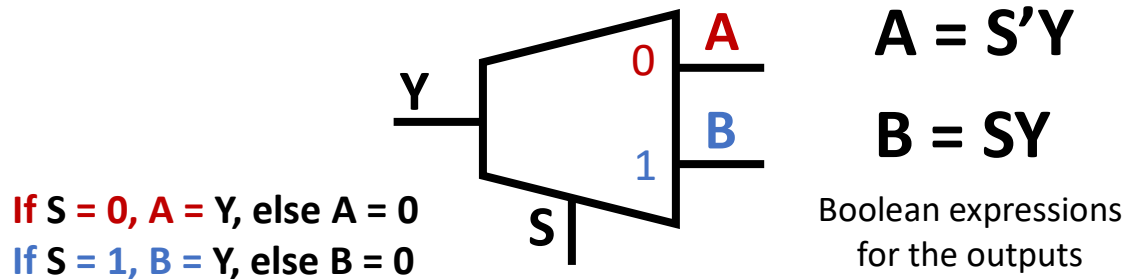
The truth table on the right describes the exact functionality. When S = 0, A copies exactly what Y is, while B remains 0. When S = 1, B copies exactly what Y is, while A remains 0.



Demultiplexers

- Demultiplexers direct the *single* input to *one of many* (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-2 demux

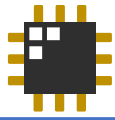


Note: Y is the input here (S is still an input), A and B are outputs

13

This logic block that we were describing is called a demultiplexer, where the functionality is somewhat the reverse of what a multiplexer is. It directs the single output to one of many outputs (2^n) corresponding to the n-bit selector.

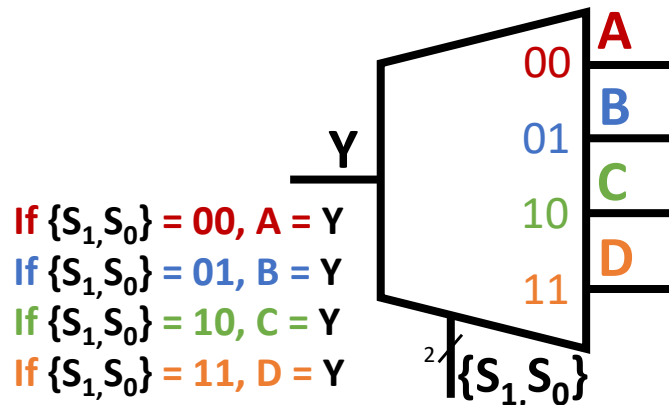
Here's the Boolean expression counterpart of the truth table shown in the previous slide. Since A copies whatever Y is when S is 0, then we have the expression $A = S'Y$. Since B copies whatever Y is when S is 1, then we have the expression $B = SY$. If the conditions are not satisfied, B remains 0 whenever S is 0 and A remains 0 whenever S is 1, which exactly corresponds to the high-level description of the logic block we had.



Demultiplexers

- Demultiplexers direct the single input to many (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



Inputs			Outputs			
S_1	S_0	Y	A	B	C	D
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

14

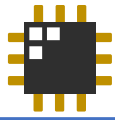
As a practice, we should be able to write the truth table of a 1-to-4 demultiplexer given the high-level functionality. The truth table is already colored for convenience, highlighting the inputs that matter given a specific condition (value of S_1 and S_0).

As long as $S_1S_0 = 00$, A copies Y, while the other outputs remain 0.

If $S_1S_0 = 01$, B copies Y, while other outputs remain 0.

If $S_1S_0 = 10$, C copies Y, while other outputs remain 0.

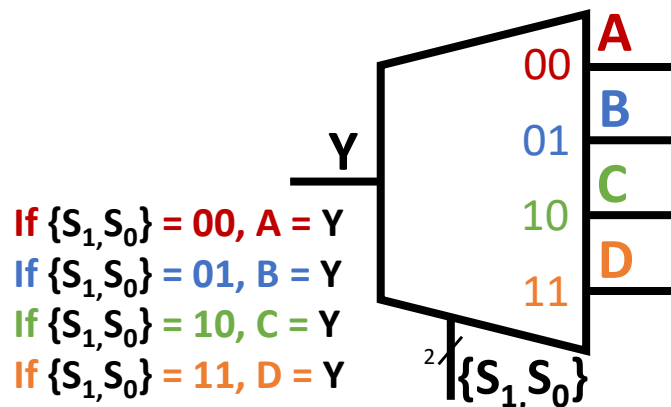
If $S_1S_0 = 11$, D copies Y, while other outputs remain 0.



Demultiplexers

- Demultiplexers direct the single input to many (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



Inputs			Outputs			
S_1	S_0	Y	A	B	C	D
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	0	0
0	1	1	0	1	0	0
1	0	0	0	0	0	0
1	0	1	0	0	1	0
1	1	0	0	0	0	0
1	1	1	0	0	0	1

15

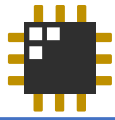
Here's the completed truth table given the high-level functionality description of a 1-to-4 demultiplexer.

As long as $S_1S_0 = 00$, A copies Y, while the other outputs remain 0.

If $S_1S_0 = 01$, B copies Y, while other outputs remain 0.

If $S_1S_0 = 10$, C copies Y, while other outputs remain 0.

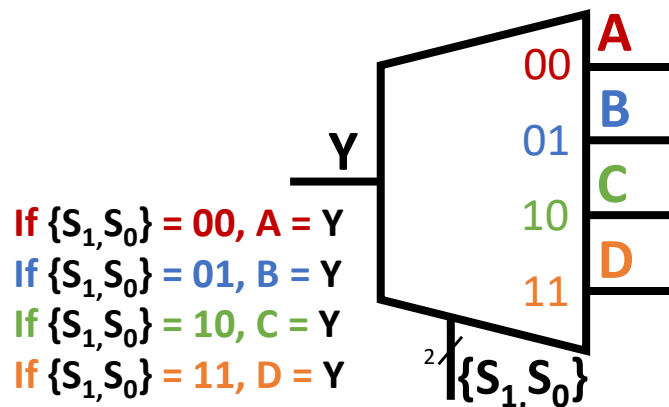
If $S_1S_0 = 11$, D copies Y, while other outputs remain 0.



Demultiplexers

- Demultiplexers direct the single input to many (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



$$A = S_1'S_0'Y$$

$$B = S_1'S_0Y$$

$$C = S_1S_0'Y$$

$$D = S_1S_0Y$$

Boolean expressions for the outputs

16

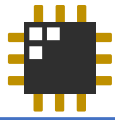
Of course, given the truth table we had earlier, we can write Boolean expressions for the output. Since we have 4 different outputs in this case, then we have 4 different expressions.

If $S_1S_0 = 00$, that implies $S_1'S_0'$, A copies whatever Y is, so the complete expression for A is $S_1'S_0'Y$.

If $S_1S_0 = 01$, that implies $S_1'S_0$, B copies whatever Y is, so the complete expression for B is $S_1'S_0Y$.

If $S_1S_0 = 10$, that implies S_1S_0' , C copies whatever Y is, so the complete expression for C is $S_1S_0'Y$.

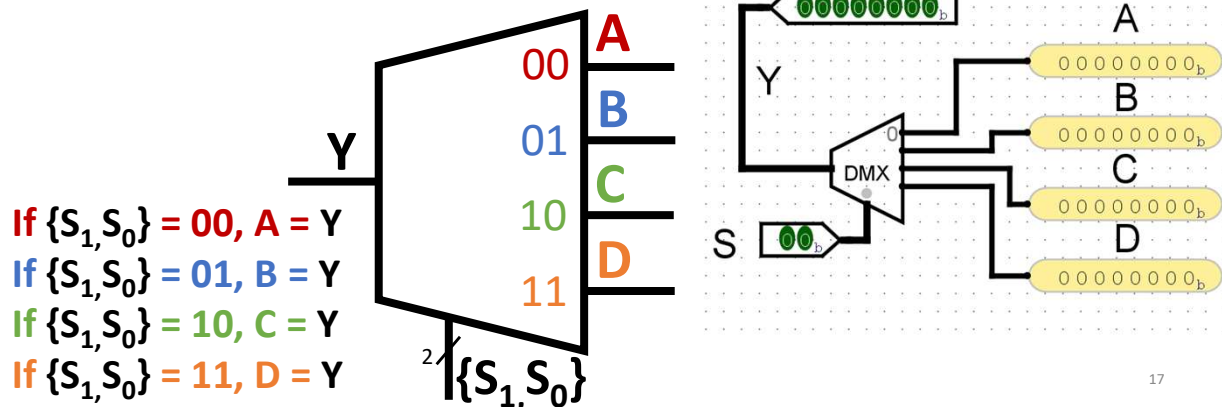
If $S_1S_0 = 11$, that implies S_1S_0 , D copies whatever Y is, so the complete expression for D is S_1S_0Y .



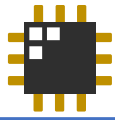
Demultiplexers

- Demultiplexers direct the single input to many (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux

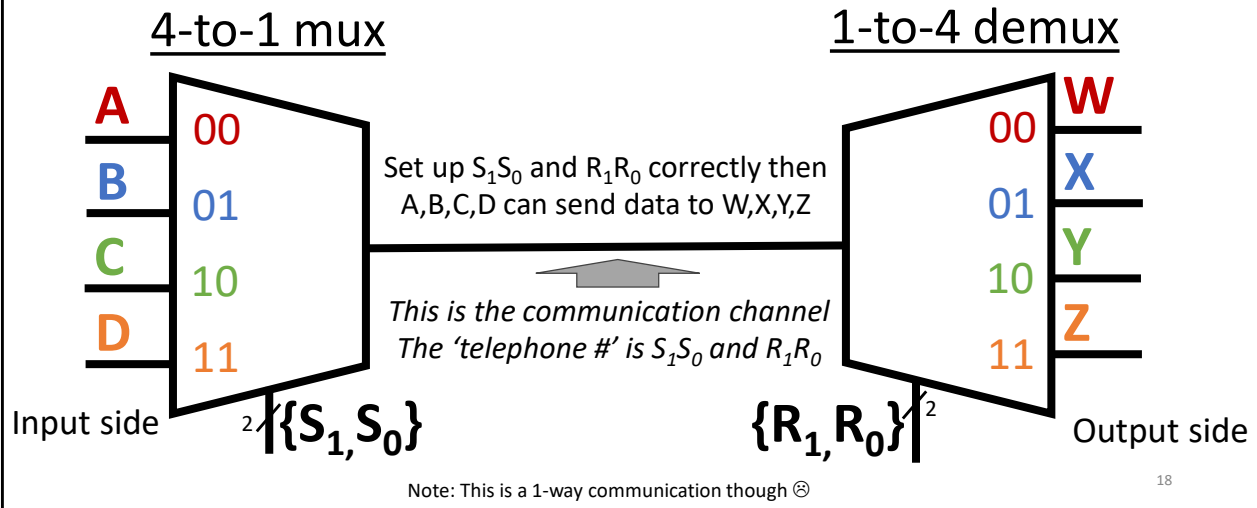


Here's the corresponding demonstration of a 1-to-4 demultiplexer. As you can see, the output Y is just being copied to one of the outputs depending on the value of the selector, while the other outputs remain 0 if not selected. The output corresponding to 00 is at the top, marked with 0, and then increases in binary value downward.

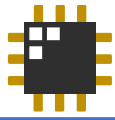


Multiplexer + Demultiplexer

- Aside: Multiplexer + Demultiplexer is like a communication channel



As a little aside, a multiplexer and a demultiplexer that is connected together can be imagined like a communication channel, like a telephone exchange system. Let's say there are multiple telephones (inputs A, B, C, D) that want to connect to multiple other telephones (outputs W, X, Y, Z). In this analogy, the 'telephone number' for any of the inputs (A, B, C, D) to contact the outputs (W, X, Y, Z) is determined by the selector inputs S_1S_0 and R_1R_0 . For instance, if $S_1S_0 = 00$, the input A is using the communication channel. Then if $R_1R_0 = 01$, then X is the one receiving the message from the input A. Given this setup, we can have 16 different combinations (2^4 , since it depends on S_1, S_0, R_1, R_0) of sender-receiver depending on the values of the 'telephone number' which corresponds to the 4 selectors.



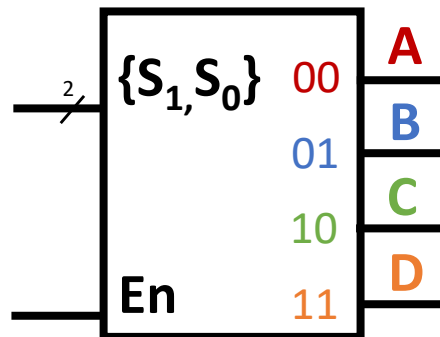
Decoders

- Demultiplexers can also be called as *decoders (with enable)*

- Drawn differently but the functionality is exactly the same!

2-to-4 decoder

The 'primary' input becomes S_1S_0 , thus the block name is 2-to-4 decoder in this example



If $\{S_1, S_0\} = 00$, $A = \text{En}$

If $\{S_1, S_0\} = 01$, $B = \text{En}$

If $\{S_1, S_0\} = 10$, $C = \text{En}$

If $\{S_1, S_0\} = 11$, $D = \text{En}$

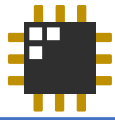
En means enable since the output is *enabled* (outputs a 1) when $\text{En} = 1$, else all outputs = 0.

19

Demultiplexers are also commonly known as decoders with enables. It is basically the same logic block but is drawn differently.

The enable input allows or denies the entry of data into the decoder. When the enable input is set to 1, the decoder works normally, and the binary data at the input is directed to the output line as determined by the select inputs.

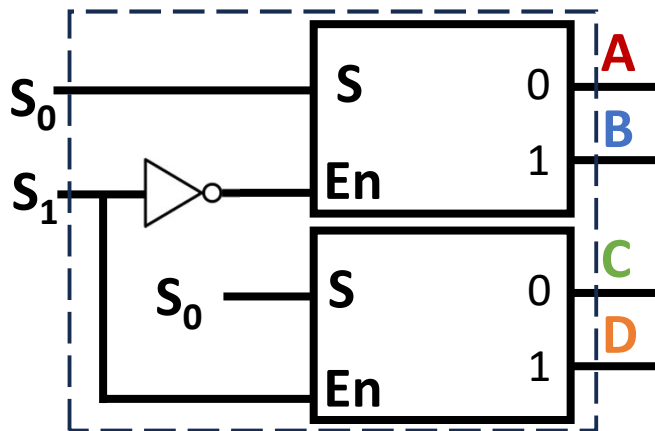
However, when the enable input is set to 0, all outputs of the decoder go to the logic 0 state, regardless of what data is coming from the input or what values the select inputs have. This is similar to the demultiplexer since the output, when selected, just copies whatever the input is, so if the input is 1 (enabled), then the selected output will copy that 1; else if the input is 0 (not enabled), then the selected output will remain 0.



Combining decoders

- You can build larger decoders using smaller ones!

2-to-4 decoder using 1-to-2 decoders



If $\{S_1, S_0\} = 00$, $A = 1$, else 0

If $\{S_1, S_0\} = 01$, $B = 1$, else 0

If $\{S_1, S_0\} = 10$, $C = 1$, else 0

If $\{S_1, S_0\} = 11$, $D = 1$, else 0

This 2-to-4 decoder does not have an enable though... can you make one?

20

Just like our exercise in building a larger multiplexer using smaller multiplexers, we can do the same of decoders. Here's a diagram of how a 2-to-4 decoder can be implemented using 1-to-2 decoders. Note how S_1 is inverted in one of the enables in the smaller decoders. This setup guarantees that only one of the smaller decoders are enabled at any given time, ensuring that all other outputs remain 0.

If $S_1S_0 = 00$, the smaller decoder at the top is enabled because $S_1' = 1$, then S_0 being 0 would select A to output a 1 while all the other outputs remain 0.

If $S_1S_0 = 01$, the smaller decoder at the top is enabled because $S_1' = 1$, then S_0 being 1 would select B to output a 1 while all the other outputs remain 0.

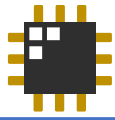
If $S_1S_0 = 10$, the smaller decoder at the bottom is enabled because $S_1 = 1$, then S_0 being 0 would select C to output a 1 while all the other outputs remain 0.

If $S_1S_0 = 11$, the smaller decoder at the bottom is enabled because $S_1' = 1$, then S_0 being 1 would select D to output a 1 while all the other outputs remain 0.

This is exactly how a 2-to-4 decoder works.

We also note that this larger 2-to-4 decoder works like it is always enabled.

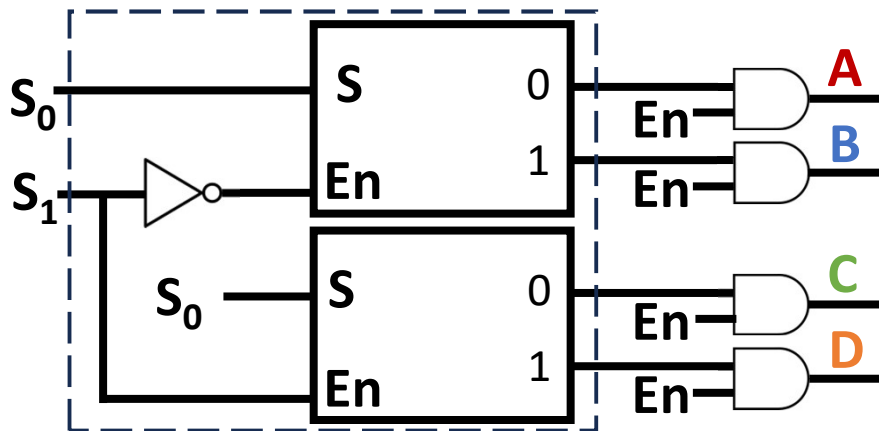
There's no enable input for this larger decoder. If we want to we can do that by putting an AND gate at the outputs of the decoder that will force them to 0 when the enable input is 0.



Combining decoders

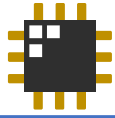
- Put an enable input for this larger decoder

2-to-4 decoder using 1-to-2 decoders



21

This diagram demonstrates that concept. By ANDing the original outputs with the En signal, then En controls whether the output can be 1 (if En = 1) or be forced to 0 (if En = 0, since anything that is ANDed with 0 is 0).



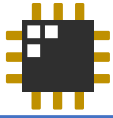
Example circuit using decoders

- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

22

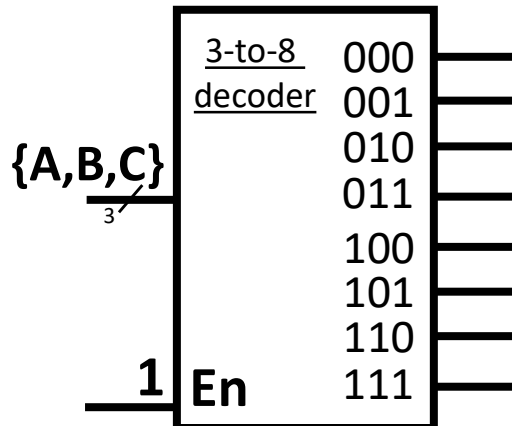
Another cool thing about decoders is that implementing the corresponding logic given a truth table is actually very easy. This is because the decoder already outputs the specific per-row combinations of a truth table. This example guides you through that process.



Example circuit using decoders

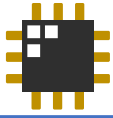
- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



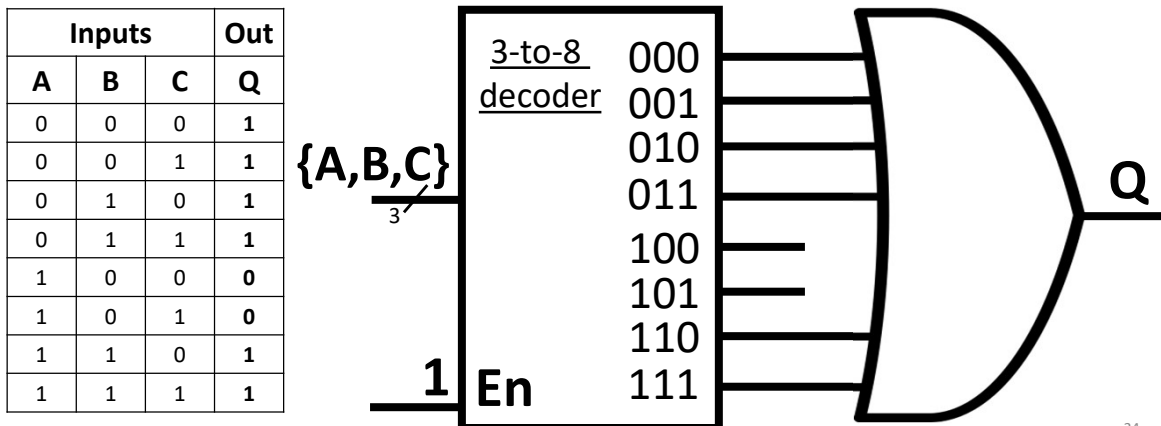
23

Since there are 3 inputs in the truth table, then this means we will have to use a 3-to-8 decoder. As you can see, there are 8 different outputs that we can select. Each of these outputs actually correspond to the different rows in the truth table that we are looking at right now. We also set En to be always 0 so that the decoder is always enabled in this case. The only inputs that matter would be the three inputs A, B, C.



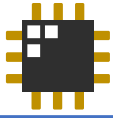
Example circuit using decoders

- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!



24

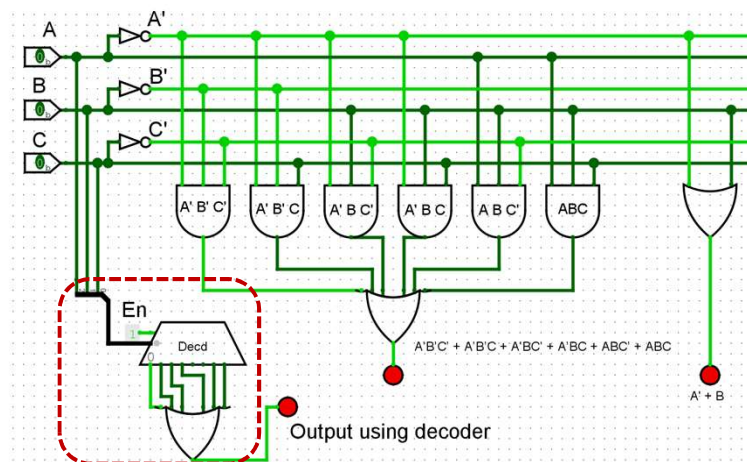
The decoder outputs become 1 when a specific condition of the inputs is satisfied. For instance, if the inputs $ABC = 000$, the first output of the decoder (at the top) is 1. That has a Boolean expression of $A'B'C'$. This is exactly how our terms would look like if we tried to analyze the truth table on its own. But now, with decoders, we can significantly speed up this process since all the input combinations are exposed as outputs. The only thing we need to do is to OR all of the relevant terms together to create the final output. The relevant terms would be those rows where the output corresponds to a 1 in the given truth table. Thus, we will end up with an implementation of a decoder and a big OR gate to combine the outputs together. Very quick and easy!



Example circuit using decoders

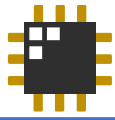
- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



25

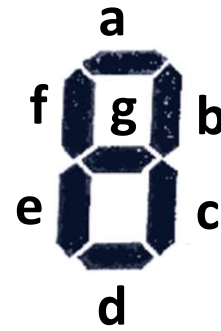
Here's a demonstration of the decoder level implementation. This is the same circuit that we had last week. The larger one was our original expression when we just analyzed the truth table. The smaller one (with only the OR gate) is the simplified expression that we came up with after utilizing Boolean algebra to find patterns and reduce the number of terms we have in the original expression. That also took us quite a bit of time to simplify. And now, using a decoder (highlighted in a red-dashed box), we can end up with the exact same functionality and we didn't take a lot of time to implement that! That's the beauty of a decoder. It allows us to implement truth tables very quickly!



Example circuit using decoders

- Fill the table for the 7-segment decoder

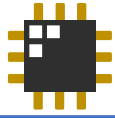
Inputs				Outputs						
W	X	Y	Z	a	b	c	d	e	f	g
0	0	0	0							
0	0	0	1							
0	0	1	0							
0	0	1	1							
0	1	0	0							
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							



Challenge: (inverted logic)
Set output to 0 if you want the LED to be on.

26

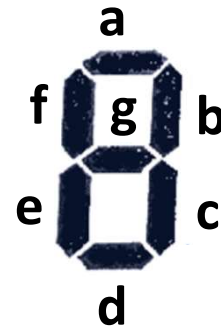
Here's another exercise. Let's say we have a seven-segment display that we want to light up with numbers 0 to 9. On the right, we are given the corresponding output assignments for each light in the seven-segment display. From this, we can start completing the table to light up the corresponding number given the 4-bit binary input W, X, Y, Z. As an additional challenge, let's use an inverted logic for this problem. This means that for a segment to light up, it must be set to 0; else, if the segment needs to be off, it must be set to 1.



Example circuit using decoders

- Fill the table for the 7-segment decoder

Inputs				Outputs						
W	X	Y	Z	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0
0	0	1	1	0	0	0	0	1	1	0
0	1	0	0	1	0	0	1	1	0	0
0	1	0	1	0	1	0	0	1	0	0
0	1	1	0	0	1	0	0	0	0	0
0	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	1	0	0

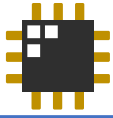


Challenge: (inverted logic)

Set output to 0 if you want the LED to be on.

27

Completing the table, we will have this. Note that we didn't exhaustively check for all the input combinations. Note how W, X, Y, Z starts with 0000 (0) and only ends with 1001 (9). Since there are four inputs, there should have been 16 different combinations all the way to 1111 (15). However, for this problem, we will assume that the inputs that will be provided is always within the range of 0-9 since that is the main purpose of a seven-segment display: to display decimal numbers.



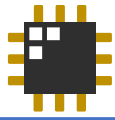
Example circuit using decoders

- Using a decoder and OR gates, implement the seven-segment decoder

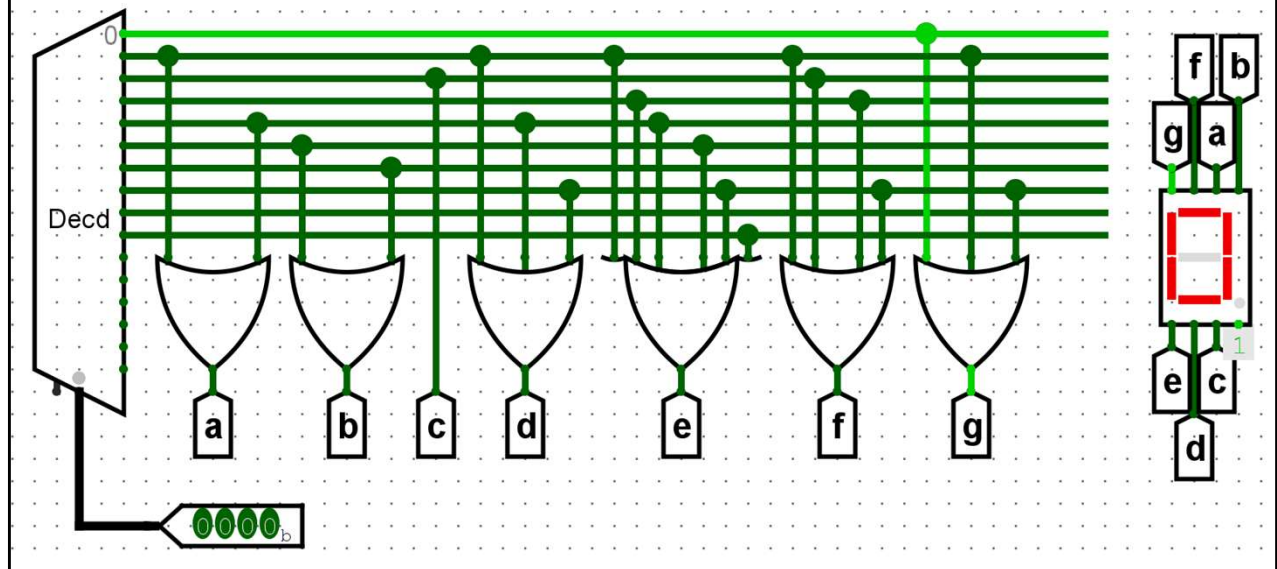
Inputs				Outputs						
W	X	Y	Z	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0
0	0	1	1	0	0	0	0	1	1	0
0	1	0	0	1	0	0	1	1	0	0
0	1	0	1	0	1	0	0	1	0	0
0	1	1	0	0	1	0	0	0	0	0
0	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	1	0	0

28

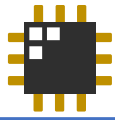
Now that we have completed the table, we can now proceed implementing the circuit using a decoder and a couple of OR gates. We have seven different outputs right now. We will have to treat them independently of each other. If you recall the process, the things that matter in this truth table is when the corresponding rows have an output of 1. For instance, the output 'a' has a 1 on input 0001 and 0100, that's it. Therefore, we will select these two outputs from our decoder and then connect them together with an OR gate. Do the same of the rest of the outputs. As you can see, the process is significantly faster since we just end up selecting which decoder outputs need to be ORed together.



Example circuit using decoders

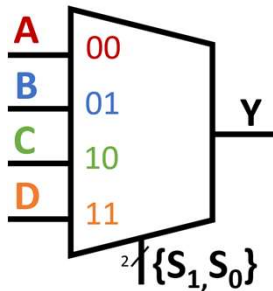


Here's a demonstration of the seven-segment display decoder, exactly following the truth table that was presented earlier. As the binary number increases from 0000 to 1001, you can see the corresponding seven-segment output on the right, consistent with the binary representation. We managed to implement this decoder very quickly with our newfound knowledge of logic blocks.



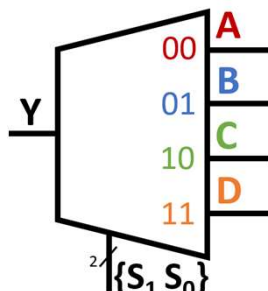
Summary

2^n -to-1
multiplexer



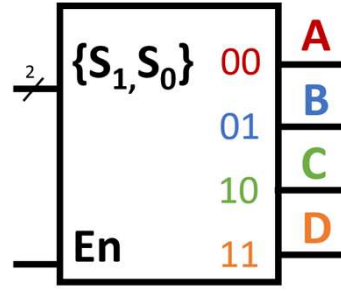
2^n inputs
n selectors
1 output

1-to- 2^n
demultiplexer



1 input
n selectors
 2^n outputs

n-to- 2^n
decoder



n inputs
1 enable
 2^n output

30

As a summary, we have just discussed three main logic blocks: a multiplexer, a demultiplexer and a decoder. As you can see, they are drawn as blocks here and the internal logic gates are not shown anymore. We are already at a higher abstraction level. The three logic blocks can be differentiated by their inputs and outputs and how they behave.

A **multiplexer**, also known as a mux, is a combinational circuit that selects binary information from one of many 2^n input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of n-bit selection lines.

A **demultiplexer**, on the other hand, is a device that takes a single input line and routes it to one of several digital output lines. A demultiplexer of 2^n outputs has n select lines, which are used to select which output line to send the input.

A **decoder** is a circuit that converts binary information from n input lines to a maximum of 2^n distinct output lines. The output lines of the decoder correspond to the different combinations of states of the input variables.